fidélitas
Universidad

fidélitas
Universidad

fidélitas
INGENIERÍA EN SISTEMAS DE COMPUTACIÓN

fidélitas
INGENIERÍA EN SISTEMAS DE COMPUTACIÓN



fidélitas
Universidad


TITULO



Colecciones

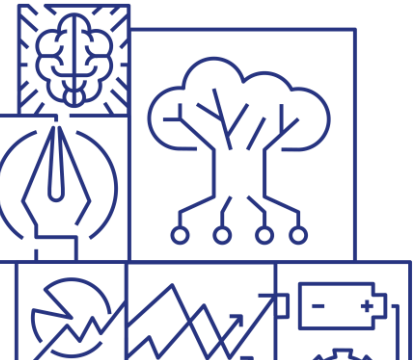
- ✓ Colecciones
- ✓ Describir las Colecciones
- ✓ Describir las implementaciones de propósito general de la interfaces principales en el marco de Colecciones
- ✓ Examinar la interfaz Map
- ✓ Examinar las clases colección tradicionales
- ✓ Utilizar colecciones genéricas

fidélitas
Universidad

Concepto de colecciones



- Según (Joyannes, 2011), “Una colección es una agrupación de objetos relacionados que forman una única entidad.”



Tipos de colecciones

Collection

- Contenedor simple de objetos no ordenados
- Los duplicados son permitidos

List

- Contenedor de elementos ordenados
- Los duplicados son permitidos

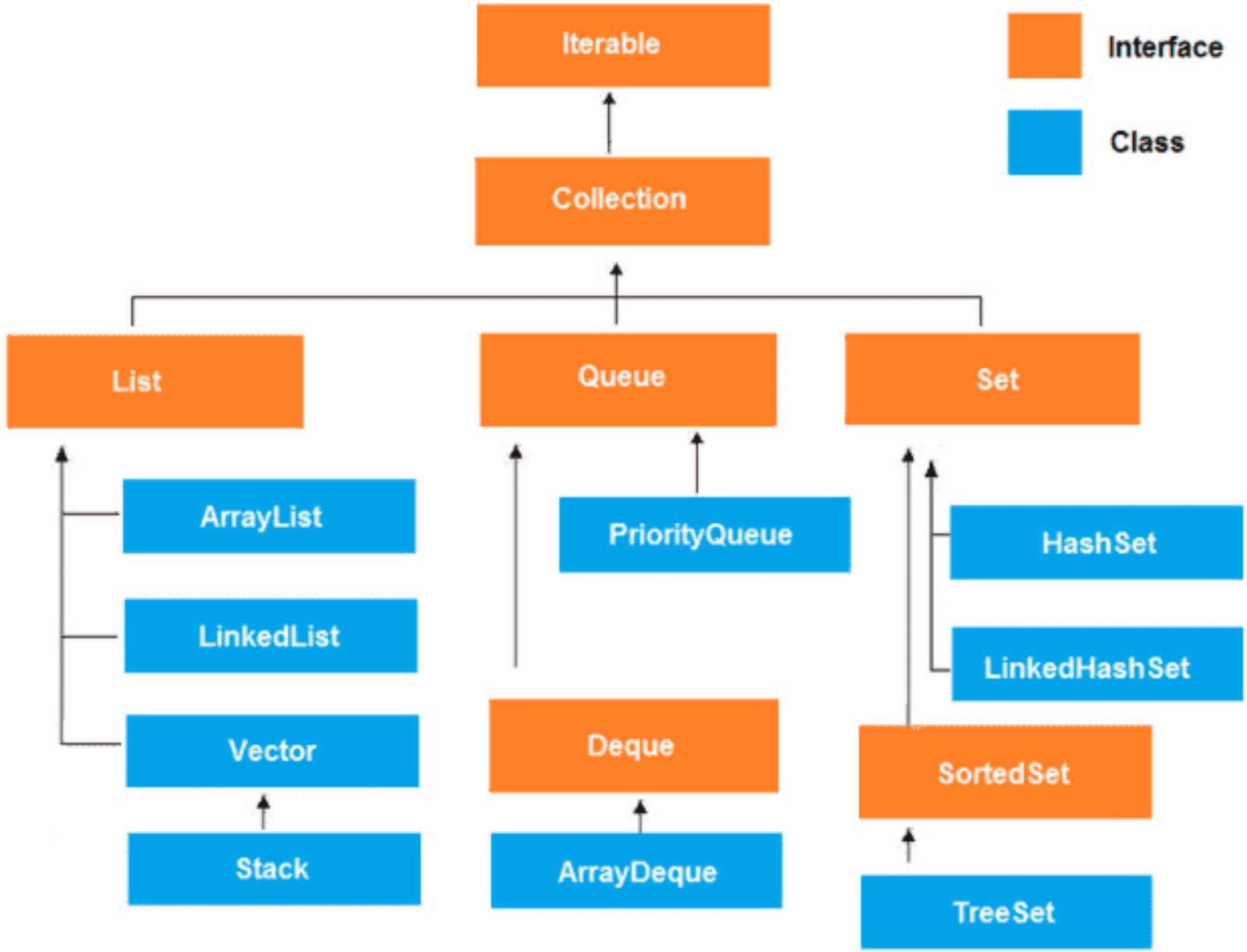
Set

- Colección desordenada de objetos
- Los duplicados no son permitidos

Map

- Colección de pares clave/valor
- La clave es utilizada para indexar el elemento
- Los duplicados no son permitidos

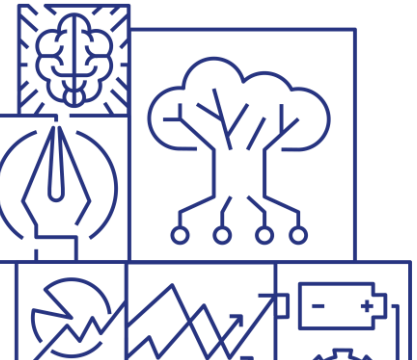
fidélitas Universidad



fidélitas
Universidad

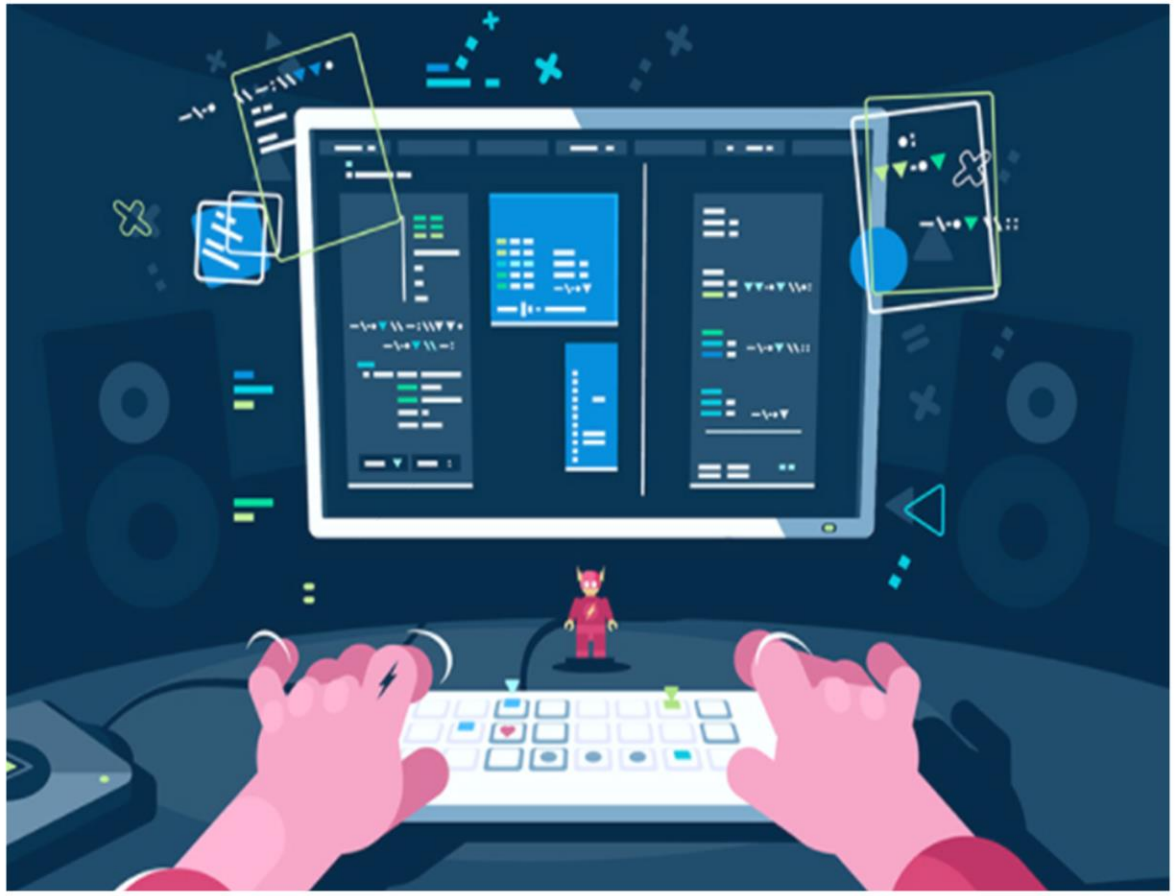
Colecciones Implementadas

	Hash Table	Resizable Array	Balanced Tree	Linked List	Hash Table + Linked List
Set	HashSet		TreeSet		LinkedHashSet
List		ArrayList		LinkedList	
Deque		ArrayDeque		LinkedList	
Map	HashMap		TreeMap		LinkedHashMap



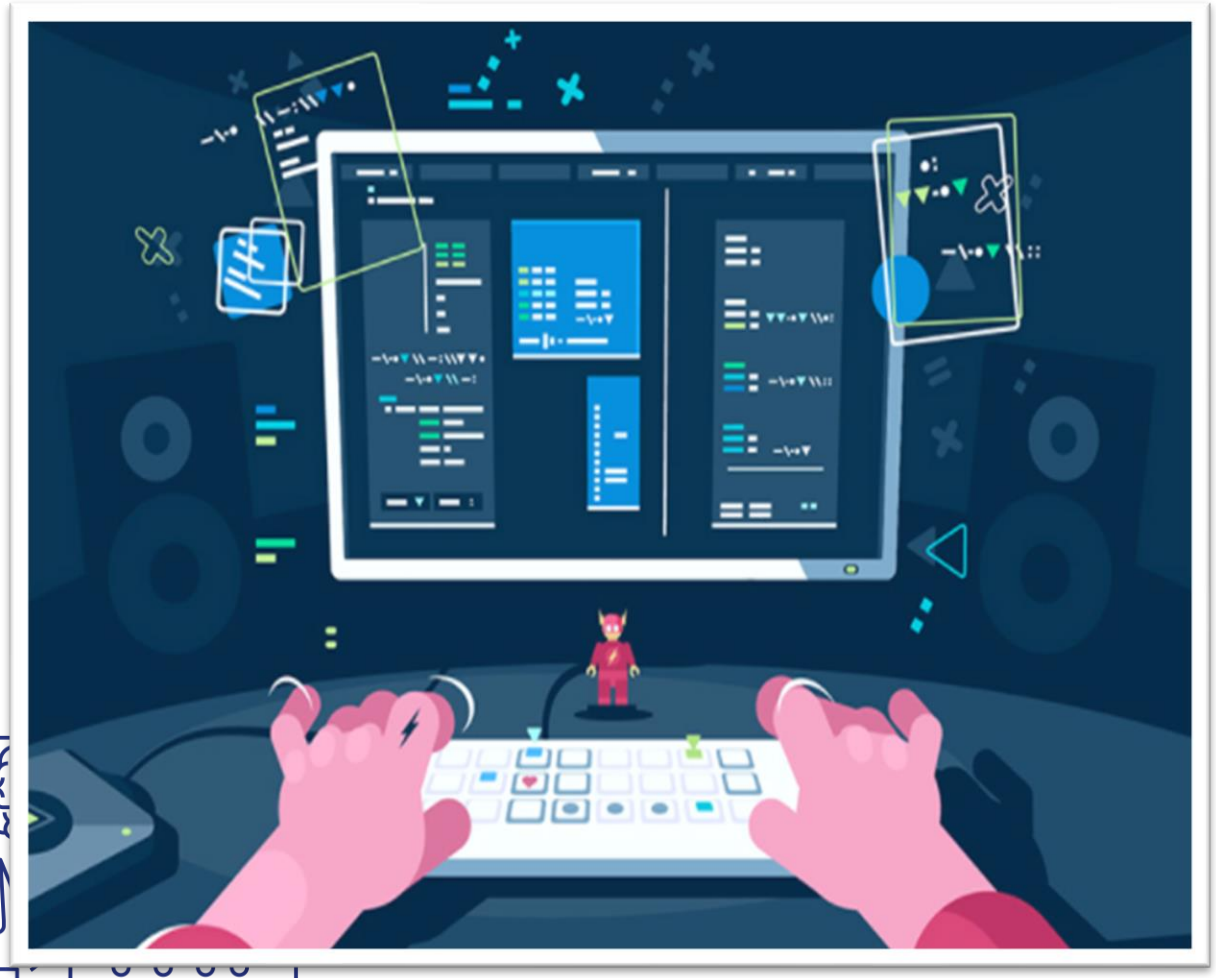
fidélitas Universidad

Un ejemplo con SET



```
Set set = new HashSet();
set.add("one");
set.add("second");
set.add("3rd");
set.add(new Integer(4));
set.add(new Float(5.0F));
set.add("second");          // duplicate, not added
set.add(new Integer(4));    // duplicate, not added
System.out.println(set);
```

Un ejemplo con LIST



```
List list = new ArrayList();
list.add("one");
list.add("second");
list.add("3rd");
list.add(new Integer(4));
list.add(new Float(5.0F));
list.add("second");         // duplicate, is added
list.add(new Integer(4));   // duplicate, is added
System.out.println(list);
```

Universidad

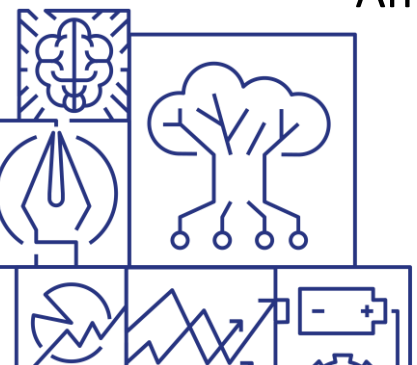
Interfaz Comparable y Comparator

Las interfaces Comparable y Comparator son útiles para ordenar colecciones:

La interfaz Comparable imparte ordenar natural clases que la implementan.

La interfaz Comparator especifica relación de orden. También puede ser utilizado para anular un ordenamiento natural

Ambas interfaces son útiles para clasificar colecciones.

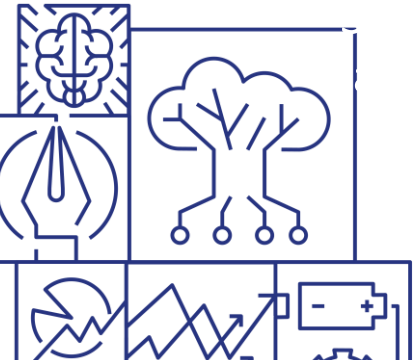


Interfaz Comparable

Se utiliza para ordenar en forma natural las clases que se implementan

El método compareTo debe codificarse para hacer comparable cualquier clase. Int compareTo (Object o1)

Los tipos de datos String, Date y Entero implementan la interfaz Comparable



Un ejemplo de Comparable



```
class Student implements Comparable {
  String firstName, lastName;
  int studentID=0;
  double GPA=0.0;
  public Student(String firstName, String lastName, int studentID,
    double GPA) {
    if (firstName == null || lastName == null || studentID == 0
    || GPA == 0.0) {throw new IllegalArgumentException();}
   this.firstName = firstName;
   this.lastName = lastName;
   this.studentID = studentID;
    this.GPA = GPA;
  }
  public String firstName() { return firstName; }
  public String lastName()  { return lastName;  }
  public int studentID() { return studentID; }
  public double GPA() { return GPA; }
```

fidélitas Universidad

Un ejemplo de Comparable



```
class Student implements Comparable {
  String firstName, lastName;
  int studentID=0;
  double GPA=0.0;
  public Student(String firstName, String lastName, int studentID,
    double GPA) {
    if (firstName == null || lastName == null || studentID == 0
    || GPA == 0.0) {throw new IllegalArgumentException();}
   this.firstName = firstName;
   this.lastName = lastName;
   this.studentID = studentID;
    this.GPA = GPA;
  }
  public String firstName() { return firstName; }
  public String lastName()  { return lastName;  }
  public int studentID() { return studentID; }
  public double GPA() { return GPA; }

  // Implement compareTo method.
  public int compareTo(Object o) {
   double f = GPA-((Student)o).GPA;
   if (f == 0.0)
    return 0;     // 0 signifies equals
   else if (f<0.0)
      return -1;    // negative value signifies less than or before
   else
      return 1;     // positive value signifies more than or after
  }
}
```

Lo probamos así:



```
TreeSet studentSet = new TreeSet();
studentSet.add(new Student("Mike", "Hauffmann",101,4.0));
studentSet.add(new Student("John", "Lynn",102,2.8 ));
studentSet.add(new Student("Jim", "Max",103, 3.6));
studentSet.add(new Student("Kelly", "Grant",104,2.3));
 Object[] studentArray = studentSet.toArray();
 Student s;
 for(Object obj : studentArray){
   s = (Student) obj;
   System.out.printf("Name = %s %s ID = %d GPA = %.1f\n",
     s.firstName(), s.lastName(), s.studentID(), s.GPA());
 }
```

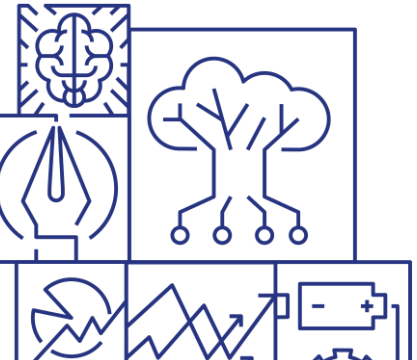
fidélitas Universidad

Interfaz Comparator

Permite clasificar en un orden diferente al natural

Se utiliza para objetos que no implementan la interface Comparable

Es necesario implementar el método de comparación compare



Otro Ejemplo



```
class Student {
  String firstName, lastName;
  int studentID=0;
  double GPA=0.0;
  public Student(String firstName, String lastName,
    int studentID, double GPA) {
    if (firstName == null || lastName == null || studentID == 0 ||
      GPA == 0.0) throw new NullPointerException();
      this.firstName = firstName;
      this.lastName = lastName;
      this.studentID = studentID;
      this.GPA = GPA;
  }
  public String firstName() { return firstName; }
  public String lastName()  { return lastName;  }
  public int studentID() { return studentID; }
  public double GPA() { return GPA; }
}
```

fidélitas Universidad

Otro Ejemplo



```
import java.util.*;
public class NameComp implements Comparator {
  public int compare(Object o1, Object o2) {
    return
      (((Student)o1).firstName.compareTo(((Student)o2).firstName));
  }
}

import java.util.*;
public class GradeComp implements Comparator {
  public int compare(Object o1, Object o2) {
   if (((Student)o1).GPA == ((Student)o2).GPA)
    return 0;
   else if (((Student)o1).GPA < ((Student)o2).GPA)
    return -1;
   else
    return 1;
  }
}
```

fidélitas Universidad

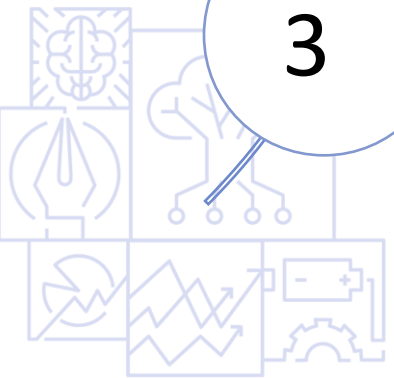
Lo probamos así



```
Comparator c = new NameComp();
TreeSet studentSet = new TreeSet(c);
studentSet.add(new Student("Mike", "Hauffmann",101,4.0));
studentSet.add(new Student("John", "Lynn",102,2.8 ));
studentSet.add(new Student("Jim", "Max",103, 3.6));
studentSet.add(new Student("Kelly", "Grant",104,2.3));
Object[] studentArray = studentSet.toArray();
Student s;
for(Object obj : studentArray) {
  s = (Student) obj;
  System.out.println("Name = %s %s ID = %d GPA = %.1f\n",
    s.firstName(), s.lastName(), s.studentID(), s.GPA());
}
```

Colecciones Genérica: ¿Qué Facilitan?

1. Proporcionan verificación de tipos en tiempo de compilación
2. Eliminar la necesidad de Casting
3. Ofrecer la posibilidad de crear en tiempo de compilación comprobación de colecciones homogéneas



fidélitas Universidad

Colecciones Genérica: ¿Cómo son?

- Colecciones genéricas:

```
ArrayList<Integer> list = new ArrayList<Integer>();
list.add(0, new Integer(42));
int total = list.get(0).intValue();
```

- Colecciones no genéricas:

```
ArrayList list = new ArrayList();
list.add(0, new Integer(42));
int total = ((Integer)list.get(0)).intValue();
```


fidélitas Universidad

¿Cómo se declaran, definen y crean?

Category	Non Generic Class	Generic Class
Class declaration	`public class ArrayList extends AbstractList implements List`	`public class ArrayList<E> extends AbstractList<E> implements List <E>`
Constructor declaration	`public ArrayList (int capacity);`	`public ArrayList<E> (int capacity);`

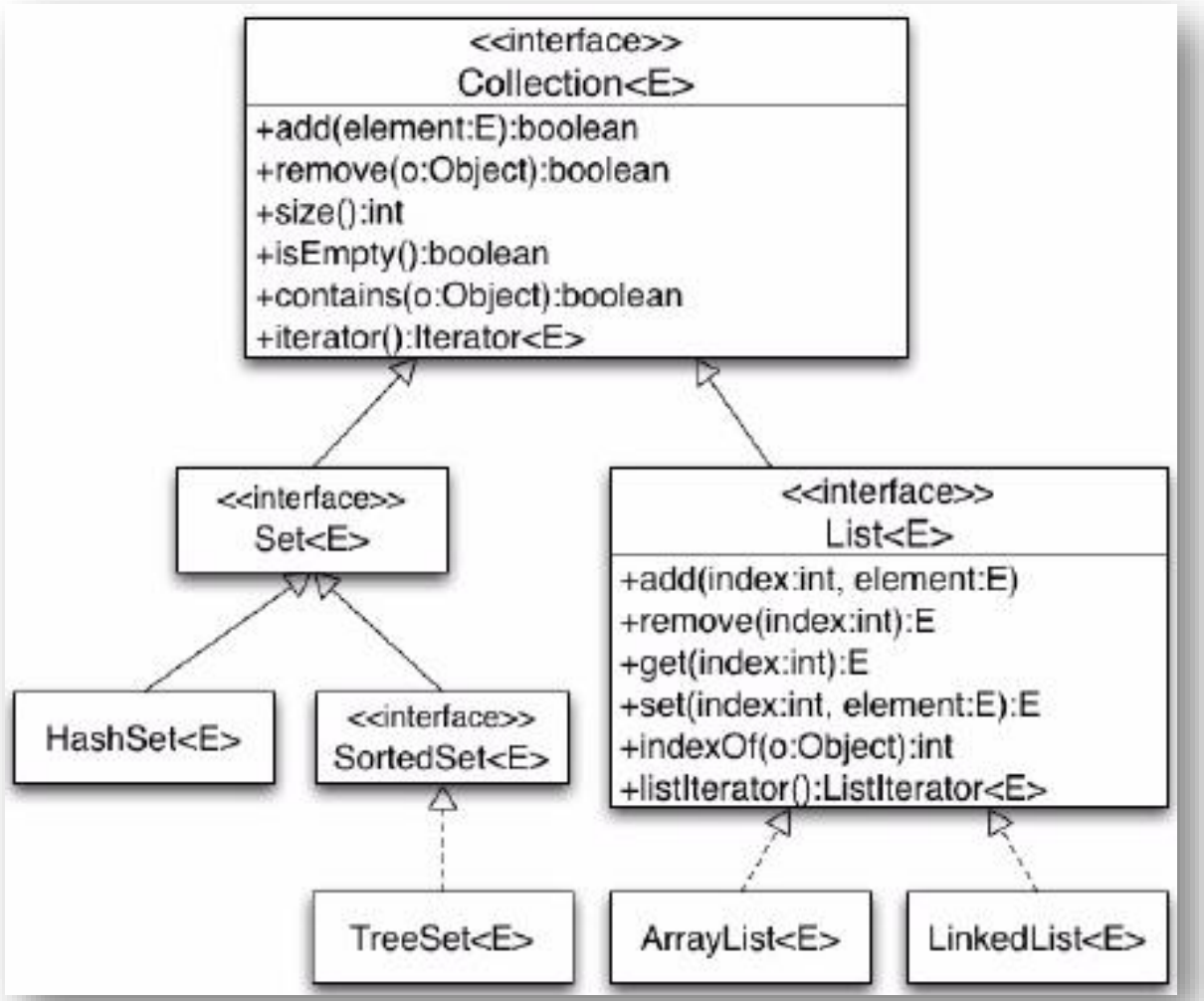
Method declaration	`public void add((Object o)`
	
	`public Object get(int index)`
		`public void add(E o)`
		
		`public E get(int index)`

Variable declaration examples	`ArrayList list1;`
	
	`ArrayList list2;`
		`ArrayList <String> list1;`
		
		`ArrayList <Date> list2;`

Instance declaration examples	`list1 = new ArrayList(10);`
	
	`list2 = new ArrayList(10);`
		`list1= new ArrayList<String> (10);`
		
		`list2= new ArrayList<Date> (10);`

fidélitas Universidad

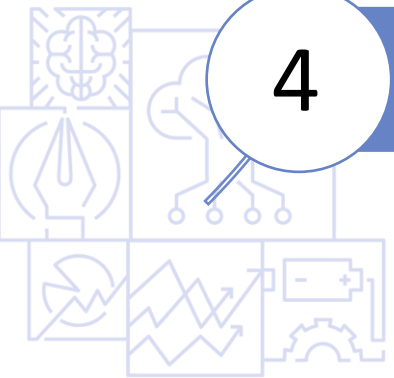
API Colecciones genéricas



fidélitas
Universidad

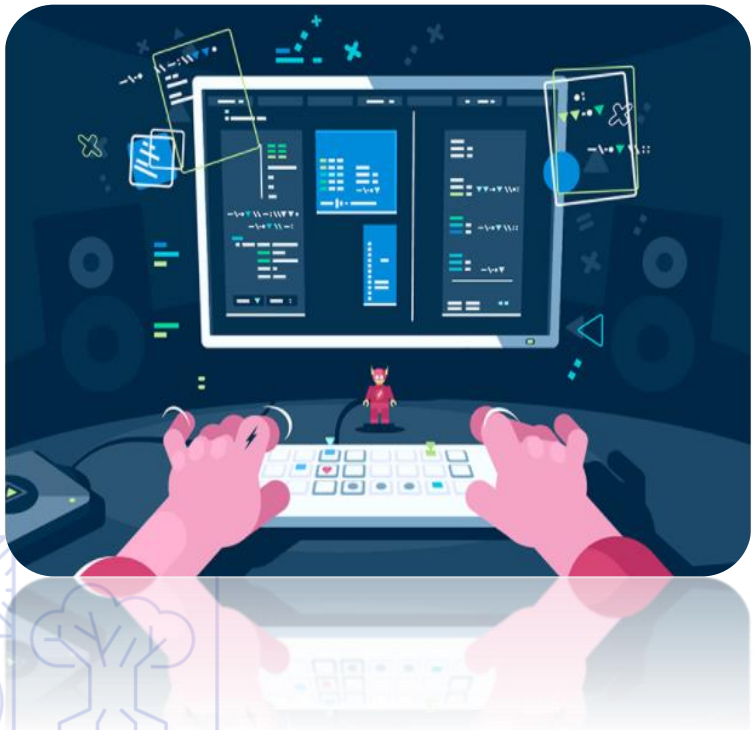
Un for mejorado… Características:

1. Iteración simplificada sobre colecciones
2. Es mucho más corto, claro y seguro
3. Utilizable con matrices
4. Fácil de implementar en ciclos anidados



fidélitas Universidad

Un ejemplo de For Mejorado



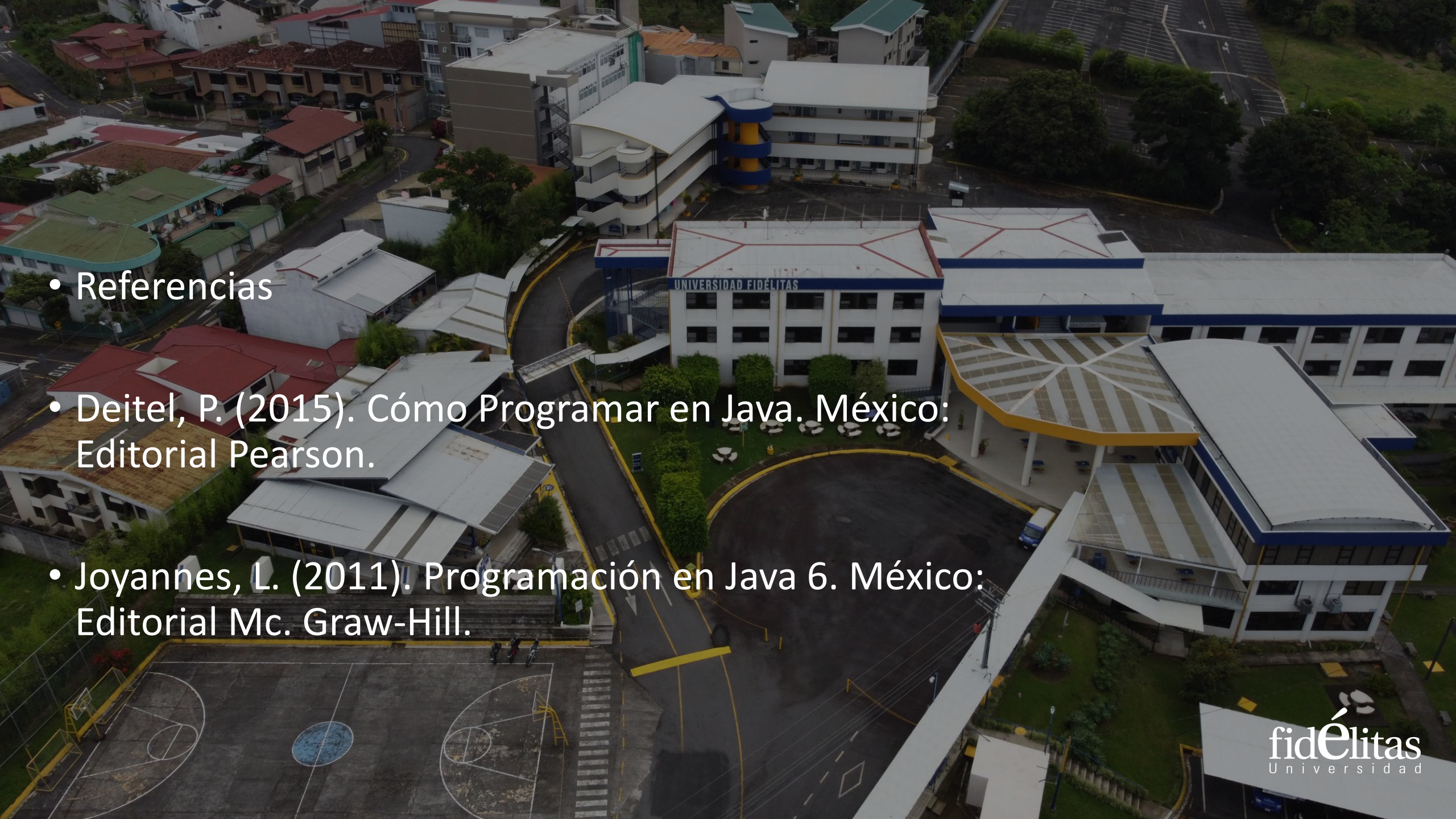
```
public void deleteAll(Collection<NameList> c){
  for ( NameList nl : c ){
    nl.deleteItem();
  }
}
```

fidélitas Universidad

Otro Ejemplo…



```
List<Subject> subjects=...;
List<Teacher> teachers=...;
List<Course> courseList = ArrayList<Course)();
for (Subject subj: subjects) {
  for (Teacher tchr: teachers) {
    courseList.add(new Course(subj, tchr));
  }
}
```

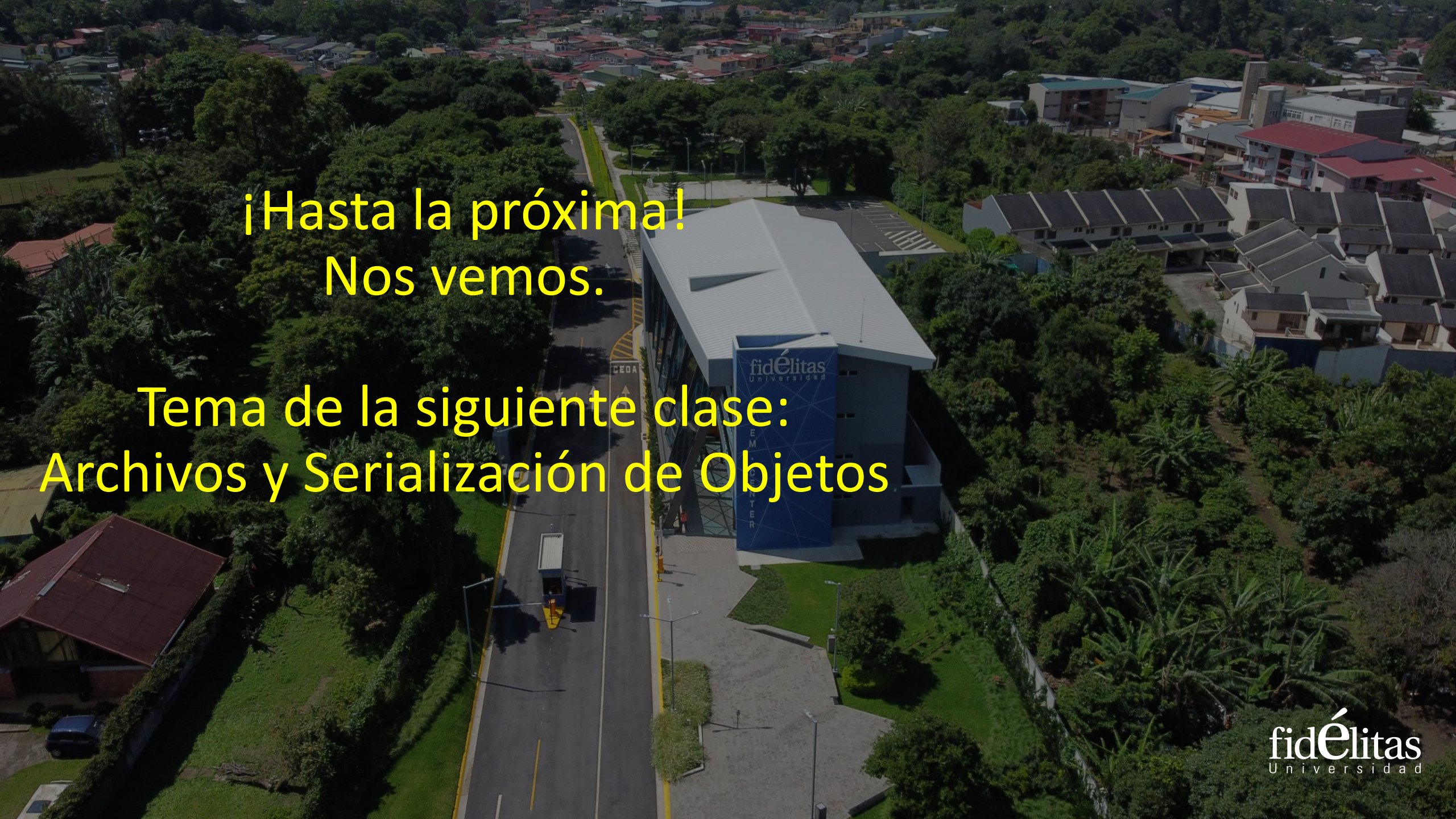


- Referencias

- Deitel, P. (2015). Cómo Programar en Java. México: Editorial Pearson.

- Joyannes, L. (2011). Programación en Java 6. México: Editorial Mc. Graw-Hill.

fidélitas Universidad


¡Hasta la próxima!
Nos vemos.
Tema de la siguiente clase:
Archivos y Serialización de Objetos
fidélitas
Universidad